

Zadanie indywidualne z Pythona

Klasa abstrakcyjna, dziedziczenie, metody statyczne i metody klasy

Temat: System obsługi spraw ZUS

Cel zadania: napisanie programu w Pythonie, który symuluje prosty system obsługi różnych typów spraw zgłaszanych do ZUS. Zadanie sprawdza rozumienie klas abstrakcyjnych, dziedziczenia, polimorfizmu, metod statycznych, metod klasy oraz funkcji zewnętrznych.

Wymagany poziom: zadanie indywidualne na ostatni dzień szkolenia, do samodzielnej implementacji.

1. Wymagania ogólne

Program ma wykorzystywać następujące elementy języka Python:

- jedną klasę abstrakcyjną,
- trzy klasy zwykle dziedziczące po klasie abstrakcyjnej,
- dwie funkcje zewnętrzne,
- jedną metodę statyczną,
- jedną metodę klasy,
- listę obiektów oraz wywołania metod na tych obiektach.

2. Klasa abstrakcyjna SprawaZUS

Utwórz klasę abstrakcyjną **SprawaZUS**. Klasa ta powinna opisywać wspólne cechy wszystkich spraw obsługiwanych w systemie.

2.1. Atrybuty klasy

Klasa powinna przechowywać następujące informacje:

- numer_sprawy,
- imie,
- nazwisko,
- pesel,
- status.

Przykładowe statusy sprawy:

```
"nowa"  
"w trakcie"  
"zakończona"
```

2.2. Metody zwykłe

W klasie abstrakcyjnej zaimplementuj dwie zwykłe metody:

Metoda	Opis działania
zmien_status(nowy_status)	Zmienia aktualny status sprawy na podany nowy status.
pokaz_dane()	Zwraca lub wyświetla podstawowe informacje o sprawie: numer, dane osoby i status.

2.3. Metody abstrakcyjne

W klasie abstrakcyjnej zadeklaruj dwie metody abstrakcyjne, które będą implementowane osobno w każdej klasie potomnej:

Metoda abstrakcyjna	Opis działania
oblicz_czas_obsługi()	Każdy typ sprawy ma inaczej liczony przewidywany czas obsługi.
opis_sprawy()	Każda sprawa powinna zwrócić własny opis tekstowy.

3. Trzy klasy dziedziczące po SprawaZUS

Utwórz trzy klasy dziedziczące po klasie abstrakcyjnej **SprawaZUS**. Każda klasa powinna posiadać własny dodatkowy atrybut oraz własną implementację metod abstrakcyjnych.

3.1. Klasa WniosekEmerytalny

Dodatkowe pole:

- lata_pracy

Zasada obliczania czasu obsługi:

- jeśli osoba ma więcej niż 35 lat pracy, czas obsługi wynosi 14 dni,
- w przeciwnym razie czas obsługi wynosi 30 dni.

3.2. Klasa WniosekRentowy

Dodatkowe pole:

- stopien_niezdolnosci

Przykładowe wartości pola:

```
"częściowa"  
"całkowita"
```

Zasada obliczania czasu obsługi:

- dla wartości "całkowita" czas obsługi wynosi 21 dni,
- dla wartości "częściowa" czas obsługi wynosi 35 dni.

3.3. Klasa WniosekZasilkowy

Dodatkowe pole:

- liczba_dni_zwolnienia

Zasada obliczania czasu obsługi:

- jeśli zwolnienie trwa do 14 dni, czas obsługi wynosi 7 dni,
- jeśli zwolnienie trwa dłużej niż 14 dni, czas obsługi wynosi 14 dni.

4. Metoda statyczna

W klasie **SprawaZUS** dodaj metodę statyczną:

```
czy_poprawny_pesel(pesel)
```

Metoda ma sprawdzać, czy PESEL:

- jest napisem,
- ma dokładnie 11 znaków,
- składa się tylko z cyfr.

Przykład użycia:

```
SprawaZUS.czy_poprawny_pesel("80010112345")
```

Oczekiwany wynik:

```
True
```

5. Metoda klasy

W klasie **SprawaZUS** dodaj metodę klasy:

```
utworz_numer_sprawy()
```

Metoda ma generować kolejny numer sprawy. Przykładowe numery:

```
ZUS/1/2026
ZUS/2/2026
ZUS/3/2026
```

Wskazówka: można użyć zmiennej klasowej, na przykład:

```
licznik_spraw = 0
```

6. Dwie funkcje zewnętrzne

Poza klasami utwórz dwie funkcje pomocnicze. Funkcje te mają pracować na liście obiektów reprezentujących sprawy.

6.1. Funkcja pokaz_wszystkie_sprawy

```
pokaz_wszystkie_sprawy(lista_spraw)
```

Funkcja ma wyświetlić dane wszystkich spraw znajdujących się na liście.

6.2. Funkcja policz_laczny_czas_obslugi

```
policz_laczny_czas_obslugi(lista_spraw)
```

Funkcja ma policzyć sumę przewidywanego czasu obsługi wszystkich spraw.

Przykład: jeśli mamy trzy sprawy o czasie obsługi:

```
14 dni
21 dni
7 dni
```

funkcja powinna zwrócić:

7. Wymagania końcowe

W programie utwórz minimum trzy obiekty:

```
wniosek1 = WniosekEmerytalny(...)
wniosek2 = WniosekRentowy(...)
wniosek3 = WniosekZasiłkowy(...)
```

Następnie wykonaj następujące kroki:

1. Dodaj utworzone obiekty do listy.
2. Wyświetl wszystkie sprawy.
3. Dla każdej sprawy pokaż opis.
4. Dla każdej sprawy pokaż przewidywany czas obsługi.
5. Zmień status jednej sprawy.
6. Wyświetl łączny czas obsługi wszystkich spraw.
7. Sprawdź poprawność numeru PESEL przy pomocy metody statycznej.
8. Wygeneruj numery spraw przy pomocy metody klasy.

8. Przykładowy wynik działania programu

Program może wypisać coś podobnego:

```
Numer sprawy: ZUS/1/2026
Osoba: Jan Kowalski
Typ sprawy: Wniosek emerytalny
Status: nowa
Przewidywany czas obsługi: 14 dni
```

```
Numer sprawy: ZUS/2/2026
Osoba: Anna Nowak
Typ sprawy: Wniosek rentowy
Status: w trakcie
Przewidywany czas obsługi: 21 dni
```

```
Numer sprawy: ZUS/3/2026
Osoba: Piotr Zieliński
Typ sprawy: Wniosek zasiłkowy
Status: nowa
Przewidywany czas obsługi: 7 dni
```

```
łączny czas obsługi wszystkich spraw: 42 dni
PESEL poprawny: True
```

9. Dodatkowe wyzwanie dla chętnych

Rozbuduj program o jeden lub kilka dodatkowych elementów:

- metodę `__str__`,
- walidację statusu sprawy,
- sortowanie spraw według czasu obsługi,
- filtrowanie spraw według statusu,

- czwartą klasę, na przykład `WniosekRehabilitacyjny`.

*Wskazówka dydaktyczna: to zadanie pokazuje nie tylko składnię klas, ale też sens architektury obiektowej.
Mały system, a już widać polimorfizm w praktyce.*